Universita di Pisa

Master's Thesis

Supervised by: Professor Alessio Giorgetti

A Heuristic Approach to Routing and Spectrum Assignment in Multi-Fiber Elastic Optical Networks: Design and Implementation in TeraFlowSDN

By: ASM Chafiullah Bhuiyan
Computer Science and Networking

Academic Year: 2025–2026

Acknowledgement

First and foremost, I would like to express my deepest gratitude to my primary supervisor, Professor Alessio Giorgetti. I am incredibly thankful for his unwavering support, his willingness to share his resources, and for granting me welcoming access to his office. His generous investment of valuable time and expertise was instrumental in allowing me to complete this extensive and complex project within the given timeframe. Furthermore, I am deeply appreciative of his provision of dedicated hardware resources, which afforded me endless opportunities to conduct rigorous testing and successfully validate my work.

Finally, words cannot adequately express my profound gratitude to my family for their immense sacrifices during my time studying abroad. I owe a special and heartfelt thank you to my children, who have had to navigate a significant part of their childhood missing their father. Your love, patience, understanding, and resilience have been my greatest source of strength and motivation throughout this journey. This thesis is as much yours as it is mine.

Abstract

Thesis Title
Master's in Computer Science and Networking

The provisioning of light-paths in flexible-grid optical networks has significantly enhanced spectrum utilization; however, the integration of topologies featuring multiple parallel links between optical devices introduces profound computational complexities in Routing and Spectrum Allocation (RSA). Specifically, when multiple transponders (muxponders) interface with the exact same degree of Reconfigurable Optical Add-Drop Multiplexer (ROADM) devices, the permutation of viable routing options increases exponentially. This geometric expansion renders standard path computation mechanisms excessively heavy and inefficient for traditional Software-Defined Networking (SDN) controllers. This routing challenge is further compounded by the stringent constraints of spectrum continuity and contiguity under the ITU-T G.694.1 standard, which dictates a 12.5 GHz minimum slot width and a high-resolution 6.25 GHz slot granularity. Because optical signals directed to the same ROADM degree cannot reuse identical frequencies without severe interference, the controller is burdened with the computationally intensive task of continuously verifying the availability of every individual 6.25 GHz slot across all potential parallel routes. To overcome these dual bottlenecks of path scalability and high-resolution spectrum mapping, this thesis introduces the ParallelOpticalController, a novel microservice engineered directly into the `TeraFlowSDN` ecosystem. Leveraging the `networkX` library for advanced topological graph modeling, the proposed architecture natively supports multi-directed graphs with parallel edges. This enables the highly efficient execution of Dijkstra's algorithm to compute the optimal shortest path, while concurrently discovering and suggesting exhaustive alternative path combinations to maximize network utilization. Furthermore, to resolve the heavy spectrum calculation overhead, the controller employs an innovative bitwise mapping algorithm that performs mathematical intersections of available flex-grid slots across sequential hops, drastically reducing the time required to identify contiguous spectrum blocks. Finally, these backend computational advancements are fully integrated with a dynamic graphical WebUI that visualizes the complex parallel-link topology, endpoint interfaces, and real-time spectrum allocations, thereby offering a highly scalable, end-to-end framework for next-generation optical network management.

Contents

1 Introduction **8**
1.1 Overview . . . 8
1.2 Motivation . . . 9
1.3 Research Objectives . . . 10
1.4 Thesis Contributions . . . 11
1.5 Organization of the Thesis . . . 12

2 Background and Literature Review **13**
2.1 Software Defined Network . . . 13
2.1.1 SDN Architecture and Principles . . . 13
2.1.2 SDN in Optical Networks . . . 13
2.2 TeraflowSDN . . . 14
2.2.1 Architecture and Ecosystem . . . 14
2.2.2 Key services . . . 14
2.3 Optical Transport Networks . . . 14
2.3.1 WDM technology . . . 14
2.3.2 ITU standards . . . 14
2.3.3 Network elements (devices) . . . 15
2.3.4 Data models (configuration) . . . 15
2.4 Routing and Spectrum Allocation . . . 15
2.4.1 Spectrum continuity and contiguity constraints . . . 15
2.4.2 Existing RSA algorithms . . . 15
2.5 Problems with Parallel Links in RSA . . . 16
2.5.1 Problem statement . . . 16
2.6 Current approaches and limitation . . . 16
2.7 Related work . . . 16

3 Problem Definition and Proposed Algorithm **17**
3.1 Problem Overview . . . 17
3.2 Formal Problem Statement . . . 17
3.3 Graph Modelling with Parallel Links (topologies) . . . 17
3.4 Proposed RSA Algorithm . . . 18
3.4.1 Bandwidth & Slot Computation . . . 18
3.4.2 Path Discovery . . . 18
3.4.3 Bitmap-Based Spectrum Allocation . . . 19
3.4.4 Slot Acquisition & Assignment . . . 19

3.5 Complexity Analysis 19

4 System Design and Implementation **21**
4.1 Overall System Architecture 21
4.2 Modifications to TeraFlowSDN 21
4.2.1 Device Service Enhancements (OCDriver, OpenROADMDriver) 21
4.2.2 Context Service & Database Schema Extensions 22
4.3 RSA Computation Module (RSATools, ITUStandards) 22
4.4 Parallel Optical Controller Microservice 23
4.4.1 Service Architecture & API Design 23
4.4.2 Graph Construction with NetworkX 23
4.5 Integration with Existing TFS Services 23
4.6 Topology & Link Management (Parallel Link Support) 23
4.7 WebUI Enhancements (Spectrum Visualization) 24

5 Testbed Setup and Deployment **25**
5.1 Physical/Virtual Infrastructure 25
5.2 Emulated Devices (Transponders, ROADMS) 26
5.3 Network Topology Configuration (tfs descriptors) 26
5.4 Deployment Procedure (TeraFlowSDN & ParallelOpticalController) . 27

6 Results and Evaluation **28**
6.1 Evaluation Methodology & Test Scenarios 28
6.2 Functional Validation 28
6.3 Computational Performance 29
6.4 Spectrum Utilization Efficiency 29
6.5 Throughput Analysis 29
6.6 Comparison with Baseline/Existing Approaches 30
6.7 Statistical Analysis 30
6.8 Discussion of Results 30

7 Conclusion and Future Work **31**
7.1 Summary of Contributions 31
7.2 Limitations 32
7.3 Future Work Directions 32

List of Figures

1.1 Number of internet users over the time 8

List of Tables

1 Introduction

1.1 Overview

The unprecedented surge in global data traffic, driven largely by the widespread deployment of 5G cellular networks and Fiber-to-the-Home (FTTH) access architectures, has fundamentally reshaped the telecommunications landscape. To satisfy this monumental and continuous demand without incurring the prohibitive costs of laying new physical fiber, network operators are compelled to maximize the transmission capacity of existing optical resources. This is primarily achieved by

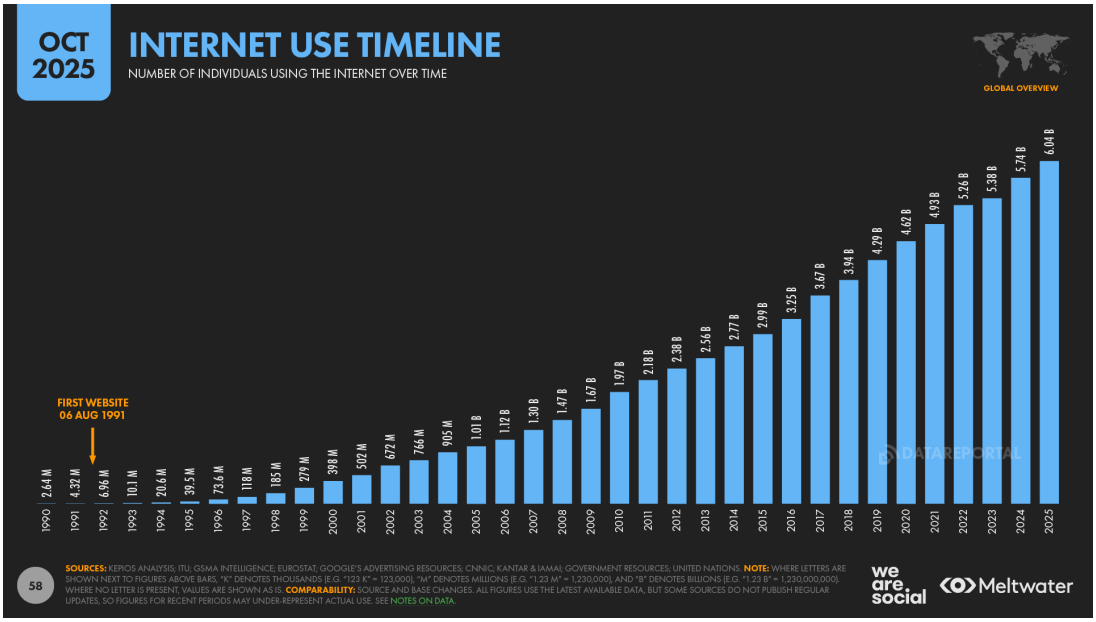

Figure 1.1: Number of internet users over the time

aggregating multiple communication channels into a single optical fiber using Wavelength Division Multiplexing (WDM) and, more specifically, Dense Wavelength Division Multiplexing (DWDM) . Concurrently, to handle the increasing complexity of these high-capacity infrastructures, the management of the optical control plane has transitioned from static, manual configurations to highly automated, centralized management systems empowered by Software-Defined Networking (SDN) [1].

As DWDM technologies have matured, the traditional fixed-grid spectrum allocation—which assigns rigid, uniform channel spacing—has become a structural bottleneck. To overcome this, flexible-grid (flex-grid) architectures have emerged as the industry standard. By adhering to ITU-T G.694.1 standards, which utilize a precise 6.25 GHz slot granularity, flex-grid networks allow for elastic, demand-driven

bandwidth allocation that vastly improves overall spectral efficiency. However, the deployment of flex-grid systems in long-haul backbone communications introduces substantial algorithmic complexity. In multi-hop networks, an optical signal must maintain both spectrum continuity (using the same frequency across all links) and spectrum contiguity (using adjacent frequency slots) from source to destination.

This Routing and Spectrum Allocation (RSA) challenge is critically exacerbated in topologies that feature multiple parallel optical links between network elements, such as adjacent Reconfigurable Optical Add-Drop Multiplexers (ROADMs) or transponders. When parallel links are introduced to a multi-hop architecture, the number of viable routing permutations increases exponentially. Consequently, traditional SDN controllers struggle to efficiently evaluate the availability of every 6.25 GHz slot across all possible path combinations, leading to severe computational bottlenecks and delayed provisioning times.

To resolve this critical scalability issue, this thesis proposes a novel approach by introducing the Parallel Optical Controller, a stateful microservice integrated directly into the TeraFlowSDN ecosystem. By leveraging advanced graph modeling techniques and a highly optimized bitwise spectrum intersection algorithm, this proposed method efficiently isolates available spectrum blocks across complex parallel routes. Ultimately, this solution effectively neutralizes the parallel link bottleneck, ensuring rapid, reliable, and highly scalable lightpath provisioning for next-generation optical networks.

1.2 Motivation

While extensive research has been conducted on Routing and Spectrum Allocation (RSA) in Elastic Optical Networks (EONs)—particularly concerning complex multi-hop lightpath provisioning—there remains a notable gap in the literature regarding the efficient management of multiple parallel optical links between adjacent devices. In traditional centralized control planes, such as those managed by other SDNs, parallel links are typically treated as isolated, individual entities rather than aggregated resources. Consequently, when a lightpath request is processed, the RSA algorithm is forced to evaluate the spectrum continuity of each parallel link independently. This legacy approach is highly inefficient, as it triggers redundant, computationally heavy, back-and-forth calculations that severely degrade the controller's performance and scalability in dense optical topologies.

The primary motivation of this thesis is to fundamentally resolve this parallel link computational bottleneck within the emerging TeraFlowSDN ecosystem. By addressing this specific algorithmic inefficiency, the research aims to design a system capable of evaluating multiple parallel paths concurrently, significantly reducing the computational overhead required to find available flex-grid spectrum.

Furthermore, theoretical RSA optimization cannot operate in a vacuum; it requires precise, real-time hardware capabilities to function effectively. Because mod-

ern optical networks are rapidly transitioning toward vendor-agnostic architectures unified by standardized data models, a crucial secondary motivation was to bridge the gap between abstract path computation and physical hardware limitations. This necessitated the extensive enhancement of OpenConfig and OpenROADM device drivers within TeraFlowSDN. By modifying these drivers to accurately extract and supply vendor-free frequency ranges and transport types, the proposed RSA algorithm is fully supported by accurate, model-driven telemetric data, ensuring the solution is both theoretically sound and practically deployable.

1.3 Research Objectives

To systematically address the challenges of Routing and Spectrum Allocation (RSA) over parallel optical links, this thesis is guided by the following core research objectives:

- **Analysis of Flexible-Grid Standards:** To investigate the operational principles of flexible-grid optical networks, specifically adhering to the ITU-T G.694.1 standards regarding spectrum slot width and 6.25 GHz granularity.
- **Evaluation of SDN Optical Intent Provisioning:** To study how optical intents and lightpath requests are currently handled by popular, industry-standard Software-Defined Networking (SDN) controllers, identifying their limitations in dense, multi-link topologies.
- **Examination of TeraFlowSDN Device Drivers:** To conduct a comprehensive study of the telemetry and configuration mechanisms within TeraFlowSDN, with a specific focus on analyzing the OpenConfig and OpenROADM device drivers.
- **Assessment of the Existing Optical Controller:** To critically evaluate the official OpticalController microservice currently operating within the TeraFlowSDN ecosystem, identifying its architectural constraints when processing complex optical intents.
- **Review of RSA Algorithmic Approaches:** To explore and analyze contemporary algorithmic strategies used to solve the RSA problem in multi-hop optical networks, evaluating their computational efficiency and spectrum assignment mechanisms.
- **Formulation of an Optimal Solution:** To synthesize the findings from the aforementioned studies and propose an optimal, highly scalable RSA framework that efficiently resolves the computational bottlenecks associated with parallel optical links.

1.4 Thesis Contributions

Building upon the established research objectives, this thesis makes several tangible contributions to the field of optical Software-Defined Networking (SDN), specifically addressing the computational bottlenecks associated with routing over parallel links. The core contributions of this work are summarized as follows:

- **Enhancement of Optical Device Drivers:** A significant extension of the existing TeraFlowSDN device drivers, specifically the OCDriver (for OpenConfig devices) and the OpenROADM driver. This contribution includes the implementation of custom filters to extract vendor-agnostic telemetry—such as minimum and maximum operating frequencies, transport types (e.g., OCH, OMS), and flexible-grid parameters—directly from XML configurations, seamlessly integrating this data into an extended Context database schema.

- **Design of an Optimized RSA Algorithm:** The formulation and implementation of a highly scalable Routing and Spectrum Allocation (RSA) algorithm tailored for topologies with multiple parallel links. This includes utilizing the networkX library to construct multi-directed graphs that accurately represent parallel edges, alongside the introduction of a novel bitmap-based bitwise intersection method designed to rapidly compute the availability of contiguous 6.25 GHz spectrum slots across multiple hops.

- **Development of the ParallelOpticalController Microservice:** The creation, deployment, and integration of a novel, stateful microservice—the ParallelOpticalController—within the TeraFlowSDN microservices architecture. This dedicated controller autonomously processes complex optical intents, executing the advanced RSA algorithm to efficiently discover primary shortest paths and alternative route combinations without overwhelming the control plane.

- **Advanced Network Visualization and WebUI Integration:** To comprehensively upgrade the TeraFlowSDN Web User Interface (WebUI) to reflect the complex physical and logical states of the optical network. This objective focuses on developing visual representations of device endpoint capabilities, real-time spectrum slot availability (based on ITU-T 6.25 GHz granularity), multi-hop topological links, and the dynamic output of the RSA computation.

By fulfilling these objectives, this research aims to deliver an end-to-end solution that spans from low-level physical device configuration up to high-level algorithmic path computation and graphical user interaction.

1.5 Organization of the Thesis

The remainder of this thesis is logically structured to guide the reader through the theoretical background, the proposed algorithmic solutions, the system implementation, and the final evaluation. The document is organized as follows:

- **Chapter 2:** Background and Literature Review provides the foundational concepts necessary to understand the research. It discusses the principles of Software-Defined Networking (SDN) in optical networks, introduces the TeraFlowSDN architecture, and details Optical Transport Networks (OTN) including WDM technology and ITU standards. The chapter concludes with a review of existing Routing and Spectrum Allocation (RSA) algorithms, highlighting the specific problems and limitations associated with parallel links.

- **Chapter 3:** Problem Definition and Proposed Algorithm formally establishes the research problem and introduces the mathematical formulation of the challenge. It details the graph modeling techniques used to represent topologies with parallel links and breaks down the proposed RSA algorithm, covering bandwidth computation, path discovery, bitmap-based spectrum allocation, and slot acquisition, followed by a thorough complexity analysis.

- **Chapter 4:** System Design and Implementation delves into the practical engineering and software architecture of the proposed solution. It outlines the modifications made to TeraFlowSDN—including device service enhancements for OpenConfig and OpenROADM, database schema extensions, and the RSA Computation Module. Furthermore, it details the development of the new Parallel Optical Controller microservice and the corresponding WebUI enhancements for spectrum visualization.

- **Chapter 5:** Testbed Setup and Deployment outlines the experimental environment used to validate the proposed system. It describes the physical and virtual infrastructure, the configuration of emulated transponders and ROADM devices, the setup of network topologies, and the deployment procedure utilizing MicroK8s.

- **Chapter 6:** Results and Evaluation presents a comprehensive analysis of the system's performance. It covers functional validation, computational performance, and spectrum utilization efficiency. The chapter also provides a throughput analysis, a comparative study against baseline approaches, and a statistical discussion of the results.

- **Chapter 7:** Conclusion and Future Work summarizes the primary contributions of the thesis, discusses the current limitations of the implemented system, and suggests potential directions for future research and development.

2 Background and Literature Review

This chapter establishes the theoretical foundation and contextual background for the research presented in this thesis. It reviews the evolution of Software-Defined Networking (SDN) in optical domains, details the architectural components of TeraFlowSDN and Optical Transport Networks (OTN), and provides a critical analysis of current Routing and Spectrum Allocation (RSA) methodologies. Finally, it identifies the limitations of existing approaches when handling parallel optical links, thereby setting the stage for the proposed solution.

2.1 Software Defined Network

2.1.1 SDN Architecture and Principles

Software-Defined Networking (SDN) represents a paradigm shift in telecommunications, decoupling the network's control logic (the control plane) from the underlying packet or optical forwarding hardware (the data plane). This separation centralizes network intelligence into a software-based controller, which maintains a global view of the network topology. The core principles of SDN include centralized management, network programmability, and the use of open, standardized APIs (such as OpenFlow or NETCONF) to dictate forwarding behaviors dynamically.

2.1.2 SDN in Optical Networks

While initially designed for packet-switched networks, SDN principles have been aggressively adapted for optical networks to create Software-Defined Optical Networks (SDON). Unlike packet networks, optical networks deal with analog physical layer impairments, wavelength continuity, and spectrum allocation. SDON abstracts these physical layer complexities, allowing the central controller to dynamically provision lightpaths, tune transceivers, and configure Reconfigurable Optical Add-Drop Multiplexers (ROADMs) automatically, moving away from historically static, manually operated optical grids.

2.2 TeraflowSDN

2.2.1 Architecture and Ecosystem

TeraFlowSDN (TFS) is a modern, cloud-native SDN controller designed to manage complex, multi-tenant optical and packet networks. Built upon a microservices architecture, TFS operates within Kubernetes environments, allowing individual control components to scale independently. This modularity makes it an ideal ecosystem for integrating custom logic, such as vendor-agnostic device drivers and novel path computation engines.

2.2.2 Key services

The TFS ecosystem relies on several critical microservices to maintain network state and execute configurations. The Context Service manages the database schemas, storing topologies, devices, endpoints, and optical link data. The Device Service handles south-bound communications, utilizing specific drivers to interact with physical or emulated hardware. Furthermore, the OpticalController acts as a dedicated service to handle optical intents, supported by a graphical WebUI for visual management and telemetry display.

2.3 Optical Transport Networks

2.3.1 WDM technology

Wavelength Division Multiplexing (WDM), specifically Dense WDM (DWDM), is the foundational technology for modern high-capacity optical networks. DWDM enables the transmission of multiple data streams over a single optical fiber by modulating each stream onto a distinct wavelength (or color) of laser light . Historically, DWDM utilized a fixed-grid spacing (e.g., 50 GHz or 100 GHz); however, modern elastic optical networks (EONs) utilize flexible-grid (flex-grid) technologies to allocate variable bandwidth depending on the exact requirements of the traffic.

2.3.2 ITU standards

The implementation of flex-grid networks is strictly governed by International Telecommunication Union (ITU) standards. Specifically, the ITU-T G.694.1 standard defines the spectral grid for DWDM applications. Under this standard, the spectrum is anchored at a central frequency of 193.1 THz. The available bandwidth is divided into minimal, indivisible frequency slots of 12.5GHz with a granularity of 6.25 GHz. Spectrum allocation must adhere to this granularity, operating across defined optical bands (such as the C, L, and S bands).

2.3.3 Network elements (devices)

The physical topology comprises two primary types of nodes: transponders (and muxponders) and ROADMs. Transponders map client-layer signals into optical channels suitable for DWDM transmission, handling modulation and forward error correction (FEC). ROADMs, conversely, are optical switches that allow individual wavelengths to be added, dropped, or passed through a node dynamically without requiring optical-to-electrical-to-optical (O-E-O) conversion. [**More to add in this section discussing about muxponders, transceivers, bandwidth capacity, roll out factors, etc. Also for the ROADM devices, it is necessary to discuss the internal architecture and assumptions we made to conduct our test.**]

2.3.4 Data models (configuration)

To achieve vendor interoperability, manufacturers relies on standardized YANG data models. OpenConfig and OpenROADM are two prominent models used to standardize the representation of optical device configurations and operational states. These models define how parameters—such as target output power, operational frequencies, and transport types (e.g., OCH, OMS)—are structured in XML or JSON formats, allowing the SDN controller to parse and push configurations uniformly regardless of the equipment manufacturer.

2.4 Routing and Spectrum Allocation

2.4.1 Spectrum continuity and contiguity constraints

Establishing a lightpath in an EON requires solving the Routing and Spectrum Allocation (RSA) problem. RSA is constrained by two strict physical requirements:

1. Spectrum Continuity: The exact same optical frequency must be maintained across all hops of a multi-hop route, provided no wavelength converters are present.

2. Spectrum Contiguity: The frequency slots assigned to a single lightpath must be adjacent to one another within the optical spectrum to form a contiguous bandwidth block.

2.4.2 Existing RSA algorithms

TBD

2.5 Problems with Parallel Links in RSA

2.5.1 Problem statement

While standard RSA algorithms perform adequately on simple topologies, their computational efficiency degrades drastically in multi-graph networks featuring multiple parallel optical links between the same adjacent devices. In such topologies, the total number of possible path combinations from a source to a destination increases exponentially. Because signals routed over the same degree of a ROADM cannot reuse identical frequencies without severe interference, the controller must exhaustively calculate and verify the availability of every 6.25 GHz slot across every parallel edge. This results in massive computational overhead and latency during lightpath provisioning.

2.6 Current approaches and limitation

Contemporary SDN controllers, such as ONOS and OpenDaylight, lack native optimizations for multi-directed parallel optical graphs. Currently, these systems treat parallel edges as entirely isolated, individual channels. When computing a path, the RSA algorithm evaluates each parallel link sequentially, executing redundant continuity and contiguity checks. This back-and-forth, iterative calculation methodology is inherently unscalable and fails to leverage the collective capacity of parallel optical pipes efficiently.

2.7 Related work

A substantial body of literature exists on optimizing RSA in Elastic Optical Networks (EONs). Previous research has extensively modeled multi-hop routing constraints, fragmentation minimization, and modulation format adaptation. However, literature addressing the specific algorithmic handling of parallel links within a centralized, vendor-agnostic SDN architecture remains sparse. Most existing works assume simple graph topologies (one link between two nodes) or rely on simplifying assumptions that violate ITU-T flex-grid parameters. Consequently, there is a distinct necessity for a novel, stateful approach—such as a dedicated bitmap-based parallel controller—to bridge this gap between theoretical RSA and practical, scalable deployment in modern multi-link infrastructures.

3 Problem Definition and Proposed Algorithm

This chapter formalizes the Routing and Spectrum Allocation (RSA) challenge within multi-graph optical topologies and presents the core algorithmic solutions developed for this thesis. It details the methodologies used to model physical networks with parallel links, compute required bandwidth and slots, discover optimal and alternative paths, and dynamically allocate spectrum using a novel bitmap intersection approach.

3.1 Problem Overview

In modern optical networks, computing lightpaths between transponders and ROADMs requires resolving the Routing and Spectrum Allocation (RSA) problem. While standard algorithms can find paths across simple topologies, significant computational bottlenecks arise when topologies feature multiple parallel links between devices. Because optical channels transmitted to the same degree of a ROADM cannot reuse identical frequencies without conflict, the controller must exhaustively calculate and track the exact spectrum slot availability across all parallel permutations. The proposed solution addresses this by introducing a parallel optical controller that utilizes advanced graph modeling and bitwise spectrum calculations to efficiently manage these parallel paths.

3.2 Formal Problem Statement

TBD (Note: I have to define the mathematical formulation to represent the problem here, such as defining the network as a graph G=(V,E), defining the sets of available frequencies, and writing the objective function to minimize spectrum fragmentation or blocking probability.)

3.3 Graph Modelling with Parallel Links (topologies)

To enable efficient path computation and expansion, the physical network topology retrieved from the TeraFlowSDN database is modeled locally into a multi-directed graph using the NetworkX Python library. Only active optical devices are considered

for this graph-building phase. Because NetworkX does not feature a default device-port-link formation, a custom mapping system was developed to link devices and endpoints. The controller generates multiple graph variations to serve different computational needs:

- `G_FREE`: A multi-directed graph that strictly includes "free" links where the optical channel status is OCH==USED[false]. This graph allows multiple parallel links between the same nodes.
- `G_simple_free`: A simplified version of the graph that filters out parallel links to find unique, distinct paths from the source to the destination.
- `G_simple`: The raw graph used to determine all possible topological paths without filtering for link utilization, which is later expanded to discover parallel combinations.

During the generation of these graphs, the controller fetches all links and creates a device-endpoint map based on TeraFlowSDN database records. For each endpoint, the channel information is retrieved to provide the algorithm with complete details regarding supported frequencies.

3.4 Proposed RSA Algorithm

3.4.1 Bandwidth & Slot Computation

Before pathing can occur, the controller must determine the exact amount of spectrum required for the requested optical intent. The bitrate for this operation is considered in GHz. The required bandwidth, B_r, is calculated using the following formula:

$$B_r = \frac{R_b}{n}(1 + \alpha)$$

where $n = log_2(M)$, and M represents the specific modulation format utilized. While several factors influence the required bandwidth, this standard formula simplifies the process for the controller. Once the bandwidth is known, the algorithm determines the number of required slots. Adhering to the minimum measurement unit defined by the ITU-T G.694.1 standard, the slot granularity is fixed at 6.25 GHz. The number of required slots, N, is calculated as:

$$N = \frac{B_r}{6.25}$$

3.4.2 Path Discovery

The path discovery phase utilizes the constructed NetworkX graphs to find viable routes. First, Dijkstra's algorithm is executed on the `G_FREE` graph to guarantee that the primary shortest path relies exclusively on unused optical links.

To discover alternative path combinations, a dedicated `find_paths()` function generates the `G_simple_free` graph, which includes all links, and executes the `all_simple_paths()` algorithm to find all possible distinct paths between the source and destination. Subsequently, an `expand_path()` function takes these lists of nodes and systematically computes all possible parallel link combinations between every two nodes along the route.

3.4.3 Bitmap-Based Spectrum Allocation

To manage the high-resolution ITU-T flex-grid granularity efficiently, the spectrum status is represented mathematically using a bitmap approach. Based on the detected optical band (e.g., C-Band, L-Band) and the corresponding minimum and maximum frequencies of the channels, the maximum bitmap value is generated using the formula:

$$bitmap - value = 2^N - 1$$

Because this produces an extremely large integer that cannot be natively stored as a numerical data type in standard relational databases like PostgreSQL, it is stored as a string and processed as an integer purely within the Python backend.

The RSA computation is handled by the `rsa_bitmap_pre_compute()` function. It processes the designated path and optical links, calculating the overall minimum and maximum frequency boundaries to generate a reference bitmap. The algorithm then performs a bitwise intersection across each consecutive hop in the path against the reference bitmap. This logical intersection rapidly identifies continuous blocks of available slots. Slots are categorized into status flags: AVAILABLE for free slots, `IN_USE` for allocated but non-restrictive fat channels (like OMS), FULL for consumed OCH endpoints, and UNAVAILABLE for slots outside the device's hardware capabilities.

3.4.4 Slot Acquisition & Assignment

Following the successful computation of an available spectrum block, the algorithm proceeds to the acquisition phase. It traverses each hop in the selected path and checks the current bitmap value stored in the `perform_rsa` cache. The algorithm then removes the alignment, converts the binary data back into an integer, and initiates a database update. The assignment is finalized by securely writing the updated state to the database via the context-client.

3.5 Complexity Analysis

`TBD` (Note: I have to write the time and space complexity of your algorithm here. For example, the Big-O notation for the networkX shortest path calculation on your

multi-graph, and the complexity of the bitwise intersection across H hops and S slots).

4 System Design and Implementation

This chapter provides a detailed technical overview of the system architecture and the specific software implementations developed to support parallel optical links. It outlines the extensive modifications made to the TeraFlowSDN ecosystem, the creation of a standalone RSA computation module, and the deployment of the novel ParallelOpticalController microservice.

4.1 Overall System Architecture

The proposed solution is built entirely within the TeraFlowSDN (TFS) microservices ecosystem. TFS operates as a cloud-native, Kubernetes-based SDN controller, making it highly modular and extensible. To solve the Routing and Spectrum Allocation (RSA) problem for parallel links, the system architecture was augmented with a newly developed ParallelOpticalController service. This service interfaces heavily with the existing ContextService (for retrieving topology and endpoint database states) and the DeviceService (for communicating with the physical and emulated optical hardware). The architecture ensures that lightpath requests are intercepted by the ParallelOpticalController, which computes the optimal routing and spectrum allocation before instructing the DeviceService to push XML configurations to the underlying transponders and ROADMs.

4.2 Modifications to TeraFlowSDN

4.2.1 Device Service Enhancements (OCDriver, OpenROADMDriver)

To perform accurate device-based spectrum filtration, the controller requires detailed hardware telemetry. The existing OCDriver (for OpenConfig devices) was heavily modified to extract device information directly from the device's XML configuration files rather than relying solely on static JSON descriptors.

1. A new device-specific filter mechanism was introduced, establishing a device_filters package containing classes like DefaultDevice.py and CNITTestROADMDevice.py.

2. The OCDriver.py was updated to execute a two-phase extraction process: it first retrieves the model_name from the device, and then maps this model to the appropriate filter class to extract transponder and port values accurately.

3. Similar enhancements were made to the OpenROADMDriver, filtering out internal mapping ports and correctly marking client ports as OCH transport types and line ports as OMS transport types.

4.2.2 Context Service & Database Schema Extensions

Because the original database lacked fields for manufacturer and hardware specifications, significant schema extensions were introduced to ContextService.

1. The DeviceModel.py table was updated to include mandatory fields: mfg_name, model, and serial_no, alongside nullable fields for hardware_version and software_version.

2. The EndPointModel.py was extended to include an index string and a transport_type. The index acts as a unique identifier (e.g., "TP1_11") generated by combining a sanitized device name and the port index, resolving previous endpoint mapping ambiguities.

3. The OpticalChannelModel was upgraded to store crucial RSA metrics: `min_frequenc` `max_frequency`, `flex_slots`, and `bitmap_value`. These schema changes were formally defined in the context.proto protocol buffers and propagated throughout the gRPC services.

4.3 RSA Computation Module (RSATools, ITUStandards)

To centralize the mathematical and logical rules of optical networking, two primary Python helper modules were developed: ITUStandards.py and RSATools.py.

- **ITUStandards.py:** This module codifies the strict ITU-T G.694.1 standards. It defines constants such as the 193.1 THz anchor frequency and the 6.25 GHz slot granularity. It also encapsulates enumerations for frequency ranges across all optical bands (O, E, S, C, L) and slot availability statuses (AVAILABLE, UNAVAILABLE, `IN_USE`).

- **RSATools.py:** Functioning as the core logic engine, this module handles the dynamic RSA calculations. It includes functions like detect_band() to identify the smallest optical band containing a given frequency, and compute_rsa_params() to determine the minimum and maximum frequencies and required flex slots. Crucially, it calculates the base hardware spectrum bitmap

using the formula $B_{\text{map}} = 2^{\text{slots}} - 1$ and dynamically generates spectrum slot tables for both routing computations and WebUI visualization.

4.4 Parallel Optical Controller Microservice

4.4.1 Service Architecture & API Design

The ParallelOpticalController was engineered as an autonomous microservice within TFS. It is containerized using Docker and deployed into the Kubernetes cluster alongside existing services. Following the deployment, both the serviceservice and webuiservice undergo rollout restarts to discover the new controller's environment variables and navigation endpoints.

4.4.2 Graph Construction with NetworkX

When an optical intent is received, the controller does not rely on static routing tables; instead, it dynamically reconstructs the network graph. Fetching endpoint and optical link data via the Context client, it utilizes the networkX package to build complex multi-directed graphs. The module `topology.py` is responsible for defining graph nodes (active optical devices) and generating distinct graph models, such as `G_FREE` for links where OCH is unused, and `G_simple` for raw path discovery.

4.5 Integration with Existing TFS Services

The controller interfaces seamlessly with the ContextService to read the current network state. During the "Slot Acquisition" phase, the controller updates the flow parameters in the database via the Context client, ensuring the allocated bitwise spectrum blocks are officially locked and recorded.

4.6 Topology & Link Management (Parallel Link Support)

A major implementation hurdle was supporting topologies with parallel links natively. The traditional method of mapping links via `endpoint_uuid` often failed when deploying topologies dynamically due to hashing mismatches. To resolve this, link descriptors were upgraded to support an `endpoint_index` fallback mechanism. Furthermore, to maintain a clean graphical user interface without overlapping link lines, parallel links are grouped and visually represented using an integer multiplier format, such as xN, detailing the exact number of parallel connections existing between two adjacent devices.

4.7 WebUI Enhancements (Spectrum Visualization)

To expose the advanced capabilities of the backend, the TFS Web User Interface underwent significant improvements.

- The device details template (detail.html) was modified to include the newly extracted Index and Transport Type metrics.
- A dedicated "Endpoint Frequency View" template was engineered, featuring a comprehensive "Frequency Capabilities Card" that displays wavelength ranges, bandwidth, and the specific optical band name.
- Most notably, the WebUI now features a real-time, scrollable "Spectrum Allocation Table". This table displays every individual 6.25 GHz slot within the device's band, utilizing color-coded badges to indicate slot status: green for AVAILABLE, yellow for `IN_USE`, gray for UNAVAILABLE (outside hardware bounds), and blue for the standard ANCHOR frequency.

5 Testbed Setup and Deployment

This chapter outlines the practical environment established to validate the proposed Routing and Spectrum Allocation (RSA) algorithms and the ParallelOpticalController. It details the configuration of the physical servers, the initialization of the Docker networking layer, the setup of emulated optical devices via XML data models, and the final microservices deployment procedure utilizing Kubernetes (MicroK8s).

5.1 Physical/Virtual Infrastructure

The experimental testbed relies on a distributed server architecture hosted at the University of Pisa (UniPi). Two primary servers, designated as Monster and Mascara, are utilized to balance the computational load between the SDN control plane and the emulated data plane.

To facilitate communication between the TeraFlowSDN controllers and the emulated devices, an isolated Docker bridge network is established. The creation of this network on the Monster server requires specifying a dedicated subnet and IP range:

```
# On secondary server
sudo docker network create --driver=bridge \
--ip-range=10.100.0.0/16 \
--subnet=10.100.0.0/16 \
-o "com.docker.network.bridge.name=brTFS" netbrTFS
```

Listing 5.1: Creating docker network

To ensure the Mascara server can route traffic to this newly created Docker subnet, static IP routing is configured. The following commands define the route through the respective gateway IP:

```
# Executed on Primary Server
sudo ip route add 10.100.0.0/16 via 131.114.54.73
route -n # Verification of the routing table
```

Listing 5.2: Creating route to the docker network

Furthermore, due to operating system constraints on Ubuntu 22.04 regarding network persistency, explicit iptables rules must be inserted on the Monster server to accept incoming traffic from the Mascara node:

```
# Executed on UniPi Server: Monster
sudo iptables -I DOCKER-USER -s 131.114.54.72 -d 10.100.0.0/16 -j ACCEPT
sudo iptables --list # Verification of firewall rules
```

Listing 5.3: Adding exceptions for teraflow packets

[This section will be written properly, because I have to introduce two setup. One for debian system with older versions of docker and the new one with ubuntu 22.04 and latest version of docker.]

5.2 Emulated Devices (Transponders, ROADMS)

Because deploying physical, large-scale flex-grid optical hardware is cost-prohibitive, the data plane is entirely simulated using advanced device emulators. These emulators accurately replicate the NETCONF/YANG interfaces of commercial optical equipment.

The configuration of these devices relies heavily on meticulously crafted XML files that define their hardware capabilities, physical ports, and operational frequencies.

- **Transponders (Muxponders):** The transponder configurations utilize the OpenConfig standard. For example, the `transponders_x4.xml` file defines a MellanoxSwitch device with multiple `QSFP56_DD_TYPE1` transceivers operating under the `ETH_400GBASE_ZR` standard. It explicitly details the optical channels, defining parameters such as the default 193.1875 THz frequency, target output power, and associated line ports.

- **ROADMs:** Reconfigurable Optical Add-Drop Multiplexers are configured to simulate specific optical degrees and frequency bands. The `roadm.xml` file dictates the input and output directional mapping of ports (e.g., ports 4101 and 5201 mapping to Degree D1). Crucially, it hardcodes the hardware bounds for the proposed RSA algorithm, explicitly defining the `MG_ON_PORT_MIN_FREQUENCY` as 193.0 THz and the `MG_ON_PORT_MAX_FREQUENCY` as 193.375 THz.

5.3 Network Topology Configuration (tfs descriptors)

To push the network topology into the TeraFlowSDN Context database, standardized JSON descriptors are utilized. These descriptors dictate the node identities, endpoint indices, and the optical links connecting them. To support the testing of topologies with multiple parallel links, the topology descriptor parsing mechanism was updated. Instead of relying strictly on unstable UUID hashing, the testbed utilises descriptors that reference the deterministic `endpoint_index` (e.g., `TP1_11`), allowing stable, repeatable topology generations with parallel edge definitions (e.g., xN multiplier configurations).

5.4 Deployment Procedure (TeraFlowSDN & ParallelOpticalController)

The entire TeraFlowSDN ecosystem, including the custom-built components, is deployed on a MicroK8s Kubernetes cluster. When deploying the novel ParallelOpticalController to a fresh TFS ecosystem, a strict five-step deployment pipeline is executed .

First, the Docker image for the new controller is built and pushed to the local container registry:

[Deployment of TeraFlowSDN]

6 Results and Evaluation

This chapter presents the performance evaluation and functional validation of the proposed Routing and Spectrum Allocation (RSA) framework. It assesses the operational success of the newly integrated ParallelOpticalController within the TeraFlowSDN (TFS) ecosystem and analyzes the efficiency of the proposed bitwise spectrum allocation algorithm across topologies featuring parallel optical links.

6.1 Evaluation Methodology & Test Scenarios

To rigorously evaluate the system, a series of test scenarios were executed within the established Docker-based emulated environment. The evaluation methodology focuses on injecting dynamic optical intents (lightpath requests) into the TeraFlowSDN controller and monitoring the system's response across multiple metrics.

The test scenarios utilize various network topologies, specifically focusing on multi-directed graphs where nodes (e.g., transponders and ROADMs) are connected via multiple parallel links, denoted by the xN multiplier format. The primary objectives of these tests are to validate the functional correctness of the device telemetry extraction, measure the computational efficiency of the path discovery algorithm, and statistically analyze the spectrum utilization under varying traffic loads.

TBD: (Detail the specific parameters of my test scenarios here, such as the exact number of nodes, the maximum number of parallel links per edge, the distribution of traffic requests, and the requested bandwidth/modulation formats used during the simulation.)

6.2 Functional Validation

Before analyzing quantitative performance, the functional correctness of the system was validated through backend logs and the enhanced Web User Interface (WebUI).

First, network connectivity and device emulation stability were verified by successfully pinging the emulated optical devices directly from the routing server (e.g., pinging 10.100.101.1 from the Mascara server). Following successful device discovery, the modified OCDriver and OpenROADM drivers successfully extracted the vendor-agnostic hardware parameters, which were confirmed via the Context Service database logs.

Furthermore, the functional validation is heavily supported by the newly developed visual representations in the TFS WebUI. When navigating to the device de-

tails page, the system successfully renders the "Endpoint Frequency View" template. This view dynamically displays the accurate Index and Transport Type of each port, alongside a comprehensive "Frequency Capabilities Card" outlining the specific optical band, wavelength range, and total flexible-grid slots.

Crucially, the "Spectrum Allocation Table" provides a real-time, color-coded visual validation of the RSA algorithm's output. The table accurately reflects the status of individual 6.25 GHz slots:

- Gray badges correctly identify UNAVAILABLE slots that fall outside the physical device's capabilities.
- Green badges represent AVAILABLE slots ready for provisioning.
- Yellow badges indicate `IN_USE` slots successfully allocated by the ParallelOpticalController.
- Blue badges accurately mark the ITU-T standard ANCHOR frequency (193.1 THz).

6.3 Computational Performance

TBD: (This section will present the execution time and computational overhead of the ParallelOpticalController. I must include graphs or tables showing the time taken to construct the NetworkX graph, the time taken to run Dijkstra's and the `all_simple_paths()` algorithms, and the speed of the bitwise spectrum intersection process as the number of parallel links increases.)

6.4 Spectrum Utilization Efficiency

TBD: (This section will evaluate how efficiently the flexible-grid spectrum is packed. I will discuss the fragmentation of the optical bands and provide metrics on how the proposed algorithm maximizes the use of 6.25 GHz slots across complex topologies compared to traditional fixed-grid or sequential parallel-link checking methods.)

6.5 Throughput Analysis

TBD: (This section will analyze the network's overall throughput based on the blocking probability. I must present data comparing the volume of successfully allocated optical intents versus rejected requests due to spectrum continuity constraints. Include an analysis of network-level

efficiency by comparing the ratio of allocated slots to the total available slots across all active links.)

6.6 Comparison with Baseline/Existing Approaches

TBD: (Here, I will benchmark your ParallelOpticalController against the default, sequential RSA mechanisms found in standard SDN controllers (like ONOS, OpenDayLight or the baseline TeraFlowSDN OpticalController). Provide comparative charts highlighting improvements in path computation time, reduction in blocking probability, and overall algorithmic efficiency when handling the xN parallel link scenarios.)

6.7 Statistical Analysis

TBD: (This section requires a deeper mathematical and statistical evaluation of the results. I will provide an analysis of the blocking paths and theoretical channel capacity against the actual used capacity during peak simulation periods.)

6.8 Discussion of Results

TBD: (I will provide a comprehensive summary and interpretation of the findings from the previous sections. Discuss whether the research objectives were met, highlight the most significant improvements brought by our parallel RSA algorithm, and acknowledge any unexpected behaviors or minor anomalies observed during the testing phase.)

7 Conclusion and Future Work

This concluding chapter summarizes the overarching achievements and technical contributions presented throughout this thesis. It critically reflects on the practical constraints and structural limitations encountered during the implementation phase and outlines potential avenues for future research to further refine the management of flexible-grid optical networks within TeraFlowSDN.

7.1 Summary of Contributions

As the demand for high-capacity telecommunications infrastructure continues to grow, optimizing the allocation of optical spectrum across complex topologies has become paramount. This thesis directly addressed the computational bottlenecks associated with Routing and Spectrum Allocation (RSA) over parallel optical links—a critical area often overlooked or inefficiently handled by traditional Software-Defined Networking (SDN) controllers.

The primary contribution of this research was the successful design, development, and deployment of the ParallelOpticalController, a stateful microservice fully integrated into the TeraFlowSDN (TFS) ecosystem. By leveraging the NetworkX library to construct multi-directed graphs, the controller natively supports and efficiently routes optical intents across topologies featuring multiple parallel edges. To resolve the heavy computational burden of high-resolution spectrum mapping, the system introduced a novel bitmap-based intersection algorithm capable of rapidly computing the availability of contiguous 6.25 GHz slots across multiple hops.

To ground this algorithmic advancement in practical hardware realities, this work significantly enhanced the TeraFlowSDN DeviceService and ContextService. The OCDriver (OpenConfig) and OpenROADM drivers were upgraded with custom model-based filters to extract vendor-agnostic telemetric data—such as operating frequencies, transport types (OCH, OMS), and hardware specifications—directly from XML configurations.

Finally, these backend capabilities were made accessible through a comprehensive upgrade to the TeraFlowSDN WebUI. The introduction of the dynamic "Spectrum Allocation Table" provides network administrators with real-time, color-coded visual feedback regarding the status (e.g., AVAILABLE, `IN_USE`, UNAVAILABLE) of individual ITU-T G.694.1 slots, drastically improving overall network transparency and operational user experience.

7.2 Limitations

TBD: More sophisticated device-based filtration, stateful characteristics of the controller, comparatively slower DB operations with intense topological intents ,generating a reply with the results and update the device with a NETCONF request.

7.3 Future Work Directions

TBD